

Cloudboosta Academy
AWS Cloud DevOps Training

CI/CD Pipeline with Azure DevOps, GitOps, and AKS

Full Project Documentation — Build Log, Steps, Errors, and Fixes

Group 2

Wisdom Chiemela Uwaga
Bolarinwa Gbenga Akinde
Ayobami Edun
Osahon Igbogbo
Emmanuel Ohuoha

May 2026

1. Project Overview

The project brief required building a fully automated CI/CD pipeline to deploy three microservices to Azure Kubernetes Service (AKS) using a GitOps model. All deployments must flow through the pipeline — no manual `kubectl` commands to deploy application changes.

1.1 Project Requirements

1. CI/CD pipeline using Azure DevOps Pipelines
2. Three microservices in different languages: Python (Flask), Node.js (Express), .NET (ASP.NET Core)
3. Container images stored in Azure Container Registry (ACR)
4. GitOps deployment model using ArgoCD
5. Kubernetes cluster: Azure Kubernetes Service (AKS) with minimum 3 nodes
6. Security gates in the pipeline: secret scanning, IaC scanning, SAST, container CVE scanning, DAST
7. Secrets management using Azure Key Vault — no passwords in code or pipeline variables
8. Self-hosted Azure DevOps agent
9. Monitoring: Prometheus and Grafana
10. All configuration stored in Git — reproducible from scratch

1.2 Architecture Summary

A developer pushes code to GitHub. Azure DevOps detects the push via webhook and runs the CI pipeline. The CI pipeline runs four security gates (Gitleaks, Checkov, Semgrep, Trivy) and if all pass, pushes Docker images to ACR tagged with the build ID. The CD pipeline then updates the Kubernetes deployment manifests in GitHub with the new image tag. ArgoCD — which continuously watches the GitHub repository — detects the manifest change and applies it to the AKS cluster. A DAST scan runs against the live endpoints after deployment.

The key design principle is the GitOps pull model: the pipeline never runs `kubectl` directly against the cluster. Instead it commits a change to Git, and ArgoCD reads that change and applies it. This means Git is always the source of truth for what is running in the cluster.

2. Technology Stack

Tool	Role	Why Chosen
Azure Kubernetes Service (AKS)	Kubernetes cluster hosting all microservices	Project requirement; enterprise standard; ARM64 nodes reduce cost
Azure DevOps Pipelines	CI/CD orchestration	Project requirement; native Azure integration
Azure Container Registry (ACR)	Private Docker image storage	Native AKS integration; no extra credentials for image pulls
Azure Key Vault	Secrets management	Secrets never in code; access audited; auto-masked in logs
ArgoCD	GitOps continuous delivery	Watches GitHub; self-healing; visual dashboard; declarative
GitHub	Source of truth for code and manifests	Public visibility; webhook integration with Azure DevOps
Gitleaks	Secret scanning	Catches accidentally committed passwords before they reach ACR
Checkov	IaC misconfiguration scanning	Detects insecure Kubernetes configs
Semgrep	SAST	Scans Python, JS, C# source code for insecure patterns
Trivy	Container CVE scanning	Scans built images for known vulnerabilities before pushing to ACR
OWASP ZAP	DAST	Sends HTTP requests to live endpoints to detect runtime vulnerabilities
Prometheus + Grafana	Monitoring	Cluster metrics, pod health dashboards, alerting
NGINX Ingress	Traffic routing	Single public IP routes to all three services via path prefix
Helm	Kubernetes package manager	Used to install NGINX Ingress and kube-prometheus-stack
Docker	Container build	Builds linux/arm64 images for ARM64 AKS nodes

3. Infrastructure Setup

3.1 Azure Login and Subscription

Every Azure operation starts with authenticating to the cloud.

Commands:

```
az login
az account set --subscription "YOUR_SUBSCRIPTION_ID"
```

Explanation:

az login opens a browser for OAuth authentication. After login, the CLI receives a token that proves identity for all subsequent commands. Subscriptions are billing containers — setting the correct one ensures resources are created in the right account.

Result:

A JSON array listing all subscriptions. The active one shows "isDefault": true.

3.2 Create Resource Group

Command:

```
az group create --name rg-gitops-project --location uksouth
```

Explanation:

A resource group is a logical container for Azure resources. All resources in this project (AKS, ACR, Key Vault, agent VM) live inside rg-gitops-project. This makes cleanup trivial — deleting the resource group deletes everything inside it.

Result:

```
{
  "name": "rg-gitops-project",
  "location": "uksouth",
  "properties": { "provisioningState": "Succeeded" }
}
```

3.3 Create Azure Container Registry (ACR)

Command:

```
az acr create \
  --resource-group rg-gitops-project \
  --name acrgitopsproject \
  --sku Basic \
  --admin-enabled true
```

Explanation:

ACR is a private Docker registry. The pipeline pushes built images here. AKS pulls images from here when deploying. The Basic SKU is sufficient for development. --admin-enabled enables username/password authentication in addition to managed identity.

Login server: acrgitopsproject.azurecr.io

3.4 Create AKS Cluster

Command:

```
az aks create \
  --resource-group rg-gitops-project \
  --name aks-gitops-cluster \
  --node-count 3 \
  --node-vm-size Standard_D2ps_v6 \
  --enable-addons monitoring \
  --generate-ssh-keys \
  --attach-acr acrgitopsproject
```

Explanation:

- --node-count 3: Three worker nodes satisfy the minimum project requirement and allow PodDisruptionBudget to permit rolling updates without downtime.
- --node-vm-size Standard_D2ps_v6: ARM64 (Ampere) VM with 2 vCPU and 4GB RAM. ARM64 was chosen because it had available quota in the uksouth region.
- --attach-acr: Creates an Azure role assignment giving the AKS managed identity AcrPull rights on ACR. Pods pull images without needing an imagePullSecret.

Duration: 5–10 minutes

3.5 Connect kubectl to the Cluster

Commands:

```
az aks get-credentials \
  --resource-group rg-gitops-project \
  --name aks-gitops-cluster
```

```
kubectl get nodes
```

Expected result (all 3 nodes Ready):

NAME	STATUS	ROLES	AGE
aks-nodepool1-xxxxxxxx-vmss000000	Ready	agent	3m
aks-nodepool1-xxxxxxxx-vmss000001	Ready	agent	3m
aks-nodepool1-xxxxxxxx-vmss000002	Ready	agent	3m

Scale cluster to 3 nodes (if created with 1):

```
az aks scale \
  --resource-group rg-gitops-project \
```

```
--name aks-gitops-cluster \  
--node-count 3 \  
--nodepool-name nodepool1
```

3.6 Create Azure Key Vault

Command:

```
az keyvault create \  
  --name kv-gitops-project \  
  --resource-group rg-gitops-project \  
  --location uksouth
```

Store ArgoCD password (after ArgoCD is installed):

```
ARGOCD_PASSWORD=$(kubectl -n argocd get secret argocd-initial-admin-secret \  
  -o jsonpath="{.data.password}" | base64 -d)
```

```
az keyvault secret set \  
  --vault-name kv-gitops-project \  
  --name argocd-admin-password \  
  --value "$ARGOCD_PASSWORD"
```

Store GitHub PAT:

```
az keyvault secret set \  
  --vault-name kv-gitops-project \  
  --name github-pat \  
  --value "ghp_XXXXXXXXXXXXXXXXXXXX"
```

4. ArgoCD — GitOps Engine

4.1 Install ArgoCD

Commands:

```
kubectl create namespace argocd
```

```
kubectl apply -n argocd \
-f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/
install.yaml
```

```
kubectl wait --for=condition=ready pod \
--all -n argocd --timeout=300s
```

Explanation:

ArgoCD runs inside the cluster as a set of pods in the argocd namespace. The install.yaml file contains Deployments, Services, RBAC rules, and Custom Resource Definitions. The wait command blocks until all pods are healthy.

4.2 Expose ArgoCD Web UI

Commands:

```
kubectl patch svc argocd-server -n argocd \
-p '{"spec": {"type": "LoadBalancer"}}'
```

```
kubectl get svc argocd-server -n argocd --watch
```

ArgoCD UI External IP: 20.162.177.70

ArgoCD UI URL: http://20.162.177.70

4.3 ArgoCD AppProject and Application

The AppProject (k8s/bootstrap/appproject.yaml) is a security boundary restricting what ArgoCD can deploy:

- Only to the microservices namespace — not cluster-wide
- Only whitelisted resource types: Deployment, Service, ServiceAccount, NetworkPolicy, ResourceQuota, LimitRange, PodDisruptionBudget, HorizontalPodAutoscaler, Ingress

The Application (argocd/application.yaml) points ArgoCD at the GitHub repository and watches the k8s/ directory with recursive sync. Automated sync policy (prune: true, selfHeal: true) means ArgoCD automatically applies changes and reverts manual drift.

Deploy commands:

```
kubectl apply -f k8s/bootstrap/appproject.yaml
kubectl apply -f argocd/project.yaml
kubectl apply -f argocd/application.yaml
```


5. Kubernetes Security Configuration

All three microservices use the same security hardening pattern. Every control is applied at both the container and pod levels.

5.1 Container Security Context

Applied to every container:

```
securityContext:
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
  capabilities:
    drop: ["ALL"]
```

Explanation:

- `allowPrivilegeEscalation: false` — prevents a process inside the container from gaining more OS permissions than it started with (e.g., via `setuid` binaries).
- `readOnlyRootFilesystem: true` — the container cannot write to its own filesystem. Any compromise that tries to write a backdoor file fails immediately. A `/tmp` directory is mounted as `emptyDir` for apps that need temporary write space.
- `drop ALL` capabilities — Linux capabilities are fine-grained superuser permissions. Dropping all of them means the container process has the absolute minimum OS access.

5.2 Pod Security Context

```
securityContext:
  runAsNonRoot: true
  runAsUser: 1001
  runAsGroup: 1001
  fsGroup: 1001
  seccompProfile:
    type: RuntimeDefault
```

- `runAsNonRoot: true` — Kubernetes refuses to start the container if the image's default user is root. Enforcement layer in addition to the Dockerfile `USER` instruction.
- `runAsUser/Group 1001` — all processes run as a dedicated unprivileged user.
- `seccompProfile RuntimeDefault` — applies the container runtime's default syscall filter, blocking ~45 dangerous system calls including `ptrace`, `mount`, and others used in container escape exploits.

5.3 NetworkPolicy (Default Deny)

A default-deny NetworkPolicy blocks all pod-to-pod and pod-to-external traffic in the microservices namespace. Individual per-service policies re-open only the specific port each service needs for incoming traffic.

If any pod is compromised, it cannot reach other pods or make outbound calls — limiting the blast radius of a breach.

5.4 PodDisruptionBudget

Each service has a PodDisruptionBudget with minAvailable: 1. This tells Kubernetes that during voluntary disruptions (node drains, rolling updates), at least one replica must remain running at all times. Combined with 2 replicas per deployment, this ensures zero-downtime deployments.

5.5 HorizontalPodAutoscaler

Each service has an HPA targeting 70% CPU utilisation, with minReplicas: 2 and maxReplicas: 6. If traffic increases, Kubernetes automatically adds pods. If traffic drops, it scales back down.

5.6 ResourceQuota and LimitRange

A ResourceQuota on the microservices namespace sets hard ceilings on total CPU, memory, and object counts. A LimitRange sets per-container defaults and bounds. Together these prevent any misbehaving workload from consuming all cluster resources.

6. CI Pipeline — Security Gates and Build

File: azure-pipelines/ci-pipeline.yml

Trigger: every push to main that touches microservices/, k8s/, or azure-pipelines/ directories.

6.1 Gate 1 — Secret Scanning (Gitleaks)

Tool: Gitleaks v8.18.4

Scans the entire git history for accidentally committed secrets (API keys, passwords, tokens, private keys).

Pipeline step:

```
docker run --rm \
  -v "$(Build.SourcesDirectory):/repo" \
  ghcr.io/gitleaks/gitleaks:v8.18.4 \
  detect \
    --source /repo \
    --redact \
    --exit-code 1 \
    --report-format sarif \
    --report-path /repo/gitleaks-report.sarif
```

The `--redact` flag replaces secret values with REDACTED in the SARIF report. The report is published as a pipeline artifact. `continueOnError: true` means the pipeline flags issues but continues (soft-fail). A hard block would be appropriate in production.

6.2 Gate 2 — IaC Scanning (Checkov)

Tool: Checkov 3.2.0

Scans Kubernetes YAML files for security misconfigurations (containers running as root, missing resource limits, privileged mode, missing health probes, etc.).

Pipeline steps:

```
pip3 install checkov==3.2.0 --quiet

export PATH="$HOME/.local/bin:$PATH"
checkov --directory k8s/ --framework kubernetes --output sarif --soft-fail \
  > checkov-results.sarif
```

6.3 Gate 3 — SAST (Semgrep)

Tool: Semgrep 1.72.0

Scans Python, JavaScript, and C# source code for insecure patterns (SQL injection, hardcoded secrets, use of unsafe functions, etc.).

Pipeline steps:

```
pip3 install semgrep==1.72.0 --quiet
export PATH="$HOME/.local/bin:$PATH"
semgrep --config=auto microservices/ --sarif \
  --output "$(Build.SourcesDirectory)/semgrep-results.sarif"
```

6.4 Build — Docker Images (linux/arm64)

The AKS nodes are ARM64. All Docker images must be built with `--platform linux/arm64`. On an amd64 build agent QEMU emulation is needed; on a native ARM64 agent it is skipped:

```
if [ "$(uname -m)" != "aarch64" ]; then
  docker run --rm --privileged multiarch/qemu-user-static --reset -p yes
else
  echo "ARM64 agent - QEMU not needed"
fi
```

```
docker build --platform linux/arm64 -t python-app:$(IMAGE_TAG)
microservices/python-app/
```

The three builds (Build_Python, Build_NodeJS, Build_DotNet) run as parallel stages to minimise total pipeline time.

6.5 Gate 4 — Container CVE Scanning (Trivy)**Tool: Trivy 0.50.4**

Scans the freshly built Docker image for known CVEs. Blocks on CRITICAL or HIGH severity, unfixed vulnerabilities.

Architecture-aware installation:

```
ARCH=$(uname -m)
if [ "${ARCH}" = "aarch64" ]; then TRIVY_ARCH="Linux-ARM64"
else TRIVY_ARCH="Linux-64bit"; fi
curl -sSL ".../trivy_0.50.4_${TRIVY_ARCH}.tar.gz" | tar -xz -C /usr/local/bin trivy
```

Scan command:

```
trivy image --exit-code 1 --severity CRITICAL,HIGH --ignore-unfixed python-app:$(IMAGE_TAG)
```

6.6 Push to ACR

Images are tagged with the pipeline build ID (`$(Build.BuildId)`), never with 'latest'. Immutable tags mean ArgoCD can detect the exact image in use, a rollback means updating the tag to a previous build ID, and two parallel pipeline runs never overwrite each other.

```
docker tag python-app:$(IMAGE_TAG) acrgitopsproject.azurecr.io/python-app:$(IMAGE_TAG)
```

```
docker push acrgitopsproject.azurecr.io/python-app:${IMAGE_TAG}
```

7. CD Pipeline — Deploy via GitOps

File: azure-pipelines/cd-pipeline.yml

Trigger: automatically when the CI pipeline completes successfully.

The CD pipeline uses the GitOps pull model — it never runs kubectl directly against the cluster.

7.1 Stage 1 — Fetch Secrets from Key Vault

```
- task: AzureKeyVault@2
  inputs:
    azureSubscription: "AZURE_SERVICE_CONNECTION"
    KeyVaultName: "kv-gitops-project"
    SecretsFilter: "argocd-admin-password,github-pat"
```

Secrets are injected as environment variables and automatically masked in Azure DevOps logs. Pipeline variables are visible to any project administrator. Key Vault access is controlled by Azure RBAC and every access is logged in Azure Monitor.

7.2 Stage 2 — Update Kubernetes Manifests

The pipeline updates the image tag in the three deployment YAML files using sed, then commits and pushes to GitHub. The commit message includes [skip ci] to prevent the CI pipeline from retriggering.

Configure git identity:

```
git config user.email "devops-bot@ci-cd-gitops.com"
git config user.name "Azure DevOps Bot [bot]"
git remote set-url origin https://x-access-token:$(github-pat)@github.com/...
```

Sync with latest GitHub main (avoids non-fast-forward push):

```
git fetch origin main
git reset --hard origin/main
```

Update image tags:

```
sed -i "s|acrgitopsproject.azurecr.io/python-app:.*|.../python-app:$(IMAGE_TAG)|g"
\
k8s/deployments/python-app-deployment.yaml
```

Commit and push:

```
git add k8s/deployments/
git commit -m "ci(cd): update image tags to build $(IMAGE_TAG) [skip ci]"
git push origin HEAD:main
```

7.3 Stage 3 — ArgoCD Sync and Health Gate

ArgoCD detects the manifest change via its polling interval, but the pipeline also triggers an immediate sync and waits for the application to be healthy.

```
argocd login $(ARGOCD_SERVER) \  
  --username admin \  
  --password $(argocd-admin-password) \  
  --grpc-web --insecure  
  
argocd app sync microservices-app --force --prune  
argocd app wait microservices-app --health --timeout 600
```

Without the health gate, the pipeline would report success as soon as the manifest was committed to Git — but pods may not have started successfully. Waiting for health means success only when the application is actually running and passing health checks.

7.4 Stage 4 — DAST (OWASP ZAP)

After deployment is healthy, ZAP runs a baseline scan against all three services via the Ingress IP.

```
INGRESS_IP="4.158.73.97"  
for APP in python nodejs dotnet; do  
  TARGET="http://${INGRESS_IP}/${APP}/"  
  docker run --rm -v "${REPORT_DIR}:/zap/wrk:rw" \  
    ghcr.io/zaproxy/zaproxy:stable zap-baseline.py \  
      -t "${TARGET}" -r "zap-${APP}.html" -J "zap-${APP}.json" -I  
done
```

ZAP findings are published as a pipeline artifact (dast-zap-results). The job has `continueOnError: true` — findings are reported but do not block deployment. A future improvement would be to set a threshold (block on CRITICAL, warn on HIGH).

8. Self-Hosted Pipeline Agent

The pipelines run on a self-hosted ARM64 VM in Azure. Using a self-hosted agent provides: native ARM64 builds (no QEMU emulation needed), pre-installed tools, and persistent Docker layer cache for faster builds.

8.1 Create the Agent VM

Commands:

```
az vm create \
  --resource-group rg-gitops-project \
  --name ado-pipeline-agent \
  --image Canonical:0001-com-ubuntu-server-jammy:22_04-lts-arm64:latest \
  --size Standard_D2ps_v6 \
  --admin-username azagent \
  --generate-ssh-keys \
  --public-ip-sku Standard
```

```
az vm open-port --resource-group rg-gitops-project \
  --name ado-pipeline-agent --port 22
```

Agent VM public IP: 20.117.146.28

Note: Canonical:0001-com-ubuntu-server-jammy:22_04-lts-arm64:latest is required for ARM64. The simpler alias Ubuntu2204 is x86-64 only and fails on ARM64 VM sizes.

8.2 Install Tools on the Agent

SSH into the VM then run:

```
sudo apt-get update
sudo apt-get install -y docker.io python3-pip python3-venv
sudo systemctl enable docker && sudo systemctl start docker
sudo usermod -aG docker azagent
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
sudo az aks install-cli
```

8.3 Register the Agent with Azure DevOps

Create an Azure DevOps PAT with scopes: Agent Pools (Read & manage), Build (Read & execute).

Register and start:

```
cd /home/azagent/agent
./config.sh \
  --url https://dev.azure.com/gitops-cicd-org \
  --auth pat \
  --token <AZURE_DEVOPS_PAT> \
  --pool Default \
  --agent ado-pipeline-agent \
```

```
| --unattended --acceptTeeEula
```

```
| sudo ./svc.sh install azagent  
| sudo ./svc.sh start
```

Both pipeline YAMLs reference pool: name: "Default" to route all jobs to this agent.

9. Monitoring — Prometheus and Grafana

9.1 Install kube-prometheus-stack via Helm

Commands:

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
```

```
kubectl apply -f k8s/bootstrap/monitoring-namespace.yaml
```

```
helm install kube-prometheus-stack prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --values k8s/monitoring/helm-values.yaml
```

This installs Prometheus, Grafana, Alertmanager, kube-state-metrics, and node-exporter as a single Helm release. All metrics are scraped automatically.

9.2 Access Grafana

```
kubectl get svc kube-prometheus-stack-grafana -n monitoring
```

Grafana IP: 85.210.216.241

Grafana URL: <http://85.210.216.241>

Default credentials: admin / admin

Built-in dashboards include: Kubernetes cluster overview, pod and container resource usage, Kubernetes API server health, and node-level OS metrics.

10. Errors Encountered and How They Were Fixed

This section documents every significant error encountered during the project, the root cause, and the fix applied.

10.1 ArgoCD AppProject Blocking PDB, HPA, and Ingress

Error:

ArgoCD sync showed "resource blocked by AppProject" for PodDisruptionBudget, HorizontalPodAutoscaler, and Ingress resources.

Root cause:

The AppProject's namespaceResourceWhitelist only listed Deployment, Service, ServiceAccount, ResourceQuota, LimitRange, and NetworkPolicy. PDB, HPA, and Ingress were not whitelisted.

Fix:

```
kubectl patch appproject microservices-project -n argocd --type=json -p='[
  {"op": "add", "path": "/spec/namespaceResourceWhitelist/-",
    "value": {"group": "policy", "kind": "PodDisruptionBudget"}},
  {"op": "add", "path": "/spec/namespaceResourceWhitelist/-",
    "value": {"group": "autoscaling", "kind": "HorizontalPodAutoscaler"}},
  {"op": "add", "path": "/spec/namespaceResourceWhitelist/-",
    "value": {"group": "networking.k8s.io", "kind": "Ingress"}}
]'
```

Then updated both argocd/project.yaml and k8s/bootstrap/appproject.yaml to persist the fix in Git.

10.2 nodejs-app Image Tag Missing from ACR

Error:

Pods showed ImagePullBackOff. kubectl describe pod showed "failed to pull image acrgitopsproject.azurecr.io/nodejs-app:55: not found".

Root cause:

CI build 55 had failed during the nodejs-app build stage after successfully pushing python-app and dotnet-app. The nodejs image was never pushed for build 55.

Fix:

Updated nodejs-app-deployment.yaml to reference build 53 (the last successful nodejs image). Later all three services were stabilised at build 57 after a clean full pipeline run.

10.3 monitoring/namespace.yaml Causing ArgoCD OutOfSync

Error:

ArgoCD reported the monitoring namespace as OutOfSync.

Root cause:

The file `k8s/monitoring/namespace.yaml` was in the `k8s/` directory watched by ArgoCD. ArgoCD tried to manage the monitoring namespace, but it was already managed by the `kube-prometheus-stack` Helm release, creating a conflict.

Fix:

Moved the file to `k8s/bootstrap/monitoring-namespace.yaml`. The `k8s/bootstrap/` directory is applied once manually and is not watched by ArgoCD, so there is no conflict.

10.4 kube-prometheus-stack CPU Scheduling Failure

Error:

Prometheus and Grafana pods were stuck in Pending. `kubectl describe pod` showed "Insufficient cpu: 0/3 nodes are available".

Root cause:

A leftover `guestbook-ui` Deployment (from an ArgoCD sample application) was running two pods consuming 200m CPU each. Total node CPU was 98% utilised, leaving no room for the monitoring stack.

Fix:

```
kubectl delete deployment guestbook-ui -n default
kubectl delete service guestbook-ui -n default
```

After removing the idle workload, the monitoring pods scheduled successfully within 2 minutes.

10.5 Standard_B2s and Standard_D2s_v5 VM Unavailable

Error:

`az vm create` failed with "The requested VM size is not available in the current region."

Root cause:

The `uksouth` region had exhausted quota for `Standard_B2s` and `Standard_D2s_v5` VM families (limit 10, already used 10 by the AKS cluster nodes).

Fix:

```
az vm list-usage --location uksouth --output table | grep -i "Standard D"
```

`Standard_D2ps_v6` (ARM64 Ampere) had available quota. Used that SKU, which also happens to be the same architecture as the AKS nodes — enabling native ARM64 builds.

10.6 ARM64 Ubuntu Image Error

Error:

az vm create failed with "The image Ubuntu2204 is not available for the requested VM size Standard_D2ps_v6."

Root cause:

The Ubuntu2204 convenience alias resolves to an x86-64 image. ARM64 VM sizes require ARM64 images.

Fix:

```
--image Canonical:0001-com-ubuntu-server-jammy:22_04-lts-arm64:latest
```

10.7 Agent Directory Empty — DNS Resolution Failed

Error:

After running the cloud-init provisioning script on the agent VM, the /agent directory was empty.

Root cause:

The cloud-init script tried to download the agent binary from vstsagentpackage.azureedge.net, but DNS resolution failed during VM initialisation. The download silently failed, leaving the directory empty.

Fix:

Downloaded the agent binary on the local machine, then SCP'd it to the VM:

```
scp vsts-agent-linux-arm64-3.248.0.tar.gz azagent@20.117.146.28:/home/azagent/
```

10.8 VS30063: You Are Not Authorized

Error:

During agent configuration (./config.sh): "VS30063: You are not authorized to access https://dev.azure.com/gitops-cicd-org."

Root cause:

The --token argument was set to the literal string YOUR_PAT_HERE instead of an actual PAT value.

Fix:

Generated a real Azure DevOps PAT with Agent Pools: Read & manage and Build: Read & execute scopes, then re-ran config.sh with the actual token value.

10.9 Agent Registered to Wrong Pool

Error:

Pipeline jobs were not being picked up. The agent was registered to the Default pool but pipeline YAMLs specified pool: name: "self-hosted-agents".

Root cause:

During the interactive `./config.sh` prompt "Enter agent pool (press enter for default)", the Enter key was pressed without typing a pool name. The agent registered to the Default pool.

Fix:

Updated all pool: entries in both `ci-pipeline.yml` and `cd-pipeline.yml` from name: "self-hosted-agents" to name: "Default" to match the actual pool.

10.10 Build 59 — Old Pipeline YAML Still in ADO

Error:

Build 59 failed with "results_kubernetes.sarif does not exist" and "connectedServiceNameARM is empty".

Root cause:

git push ado main had silently failed. The ADO Git repository still had the old pipeline YAML. Azure DevOps was executing the old file that referenced outdated artifact filenames.

Fix:

```
git remote add ado https://gitops-cicd-org@dev.azure.com/gitops-cicd-org/gitops-cicd/_git/cicd-azure-gitops-aks
git push ado main
```

10.11 pip Not Found on ARM64 Agent (Exit Code 127)

Error:

Build 60, IaC Scan stage: "pip: command not found" with exit code 127.

Root cause:

Ubuntu 22.04 minimal installation does not include pip. The package `python3-pip` must be installed separately.

Fix:

```
ssh azagent@20.117.146.28 "sudo apt-get install -y python3-pip python3-venv"
```

Also updated pipeline steps to use `pip3` and added `export PATH="$HOME/.local/bin:$PATH"` before running `checkov` and `semgrep` (`pip3 install --user places` binaries in `~/.local/bin`).

10.12 QEMU Exit Code 255 on ARM64 Agent

Error:

Build 60, Build_Python stage: "Set up QEMU for ARM64 cross-compilation" Bash exited with code '255'.

Root cause:

On a native ARM64 agent, docker run multiarch/qemu-user-static fails because QEMU is not needed and the QEMU static binary itself is an x86-64 binary that cannot run on ARM64.

Fix:

```
if [ "$(uname -m)" != "aarch64" ]; then
    docker run --rm --privileged multiarch/qemu-user-static --reset -p yes
else
    echo "ARM64 agent - QEMU not needed for linux/arm64 builds"
fi
```

10.13 Trivy Binary Incompatible with ARM64

Error:

Trivy installation failed because the download URL pointed to trivy_0.50.4_Linux-64bit.tar.gz — an x86-64 binary.

Fix:

```
ARCH=$(uname -m)
if [ "${ARCH}" = "aarch64" ]; then TRIVY_ARCH="Linux-ARM64"
else TRIVY_ARCH="Linux-64bit"; fi
curl -sSL ".../trivy_${TRIVY_VERSION}_${TRIVY_ARCH}.tar.gz" | tar -xz -C
/usr/local/bin trivy
```

10.14 continueOnError Not Valid at Stage Level

Error:

CD pipeline failed with "Unexpected value" at line 205, col 5 (YAML schema validation error).

Root cause:

continueOnError: true is a valid property for jobs and steps, but NOT for stages in Azure DevOps YAML.

Fix:

Removed continueOnError: true from the stage level. The job-level continueOnError: true was already present and sufficient.

10.15 ArgoCD CLI Downloading amd64 Binary on ARM64 Agent

Error:

CD pipeline ArgoCD sync stage: exit code 127 on "Install ArgoCD CLI". The step downloaded argocd-linux-amd64 which is an x86-64 binary.

Fix:

```
ARCH=$(uname -m)
if [ "${ARCH}" = "aarch64" ]; then ARGOCD_ARCH="arm64"
else ARGOCD_ARCH="amd64"; fi
curl -sSL -o /tmp/argocd ".../argocd-linux-${ARGOCD_ARCH}"
sudo mv /tmp/argocd /usr/local/bin/argocd
sudo chmod +x /usr/local/bin/argocd
```

11. End-to-End Test Results

11.1 CI Pipeline — Final Successful Run

Build number: 63 — triggered by commit: "fix: make QEMU/Trivy ARM64-aware, fix pip3 PATH on self-hosted agent"

Stage	Result
Gate 1 — Secret Scanning (Gitleaks)	Passed
Gate 2 — IaC Scanning (Checkov)	Passed
Gate 3 — SAST (Semgrep)	Passed
Build · Scan · Push — python-app	Passed (Trivy: no CRITICAL/HIGH unfixed CVEs)
Build · Scan · Push — nodejs-app	Passed
Build · Scan · Push — dotnet-app	Passed

Total CI duration: approximately 10 minutes

11.2 CD Pipeline — Final Successful Run

Triggered automatically by CI build 63.

Stage	Result
Fetch Runtime Secrets from Key Vault	Passed
Update K8s Manifests	Passed — committed new image tags to GitHub
ArgoCD Sync + Health Gate	Passed — all pods healthy within timeout
Gate 4 — DAST (OWASP ZAP)	Passed — findings reported as artifact

11.3 Live Application Verification

All three services accessible via NGINX Ingress at 4.158.73.97:

```
curl http://4.158.73.97/python/  
Result: {"service": "python-app", "status": "running", "version": "1.0.0"}
```

```
curl http://4.158.73.97/nodejs/  
Result: {"service": "nodejs-app", "status": "running", "version": "1.0.0"}
```

```
curl http://4.158.73.97/dotnet/  
Result: {"service": "dotnet-app", "status": "running", "version": "1.0.0"}
```

```
curl http://4.158.73.97/python/health  
Result: {"status": "healthy"}
```

11.4 GitOps Self-Healing Verified

Test:

```
kubectl scale deployment python-app --replicas=0 -n microservices
```

Result:

Within approximately 3 minutes, ArgoCD detected the drift and automatically restored 2 replicas. The cluster always matches what GitHub says.

12. Key Learnings

11. ARM64 architecture requires attention at every tool boundary. Every binary download (Trivy, ArgoCD CLI), every Docker build flag, and every base image reference must account for CPU architecture. A pipeline that works on x86-64 will fail in several non-obvious ways on ARM64.
12. GitOps requires the Git repository to be the single source of truth. Any manual change to the cluster (kubectl scale, kubectl edit) is reverted by ArgoCD. This is a feature — it enforces discipline and makes cluster state auditable.
13. Security scanning must happen before building, not after. Checking for secrets in source code and misconfigurations in YAML files before spending CI minutes on a Docker build saves time and prevents wasted pushes to ACR.
14. Secrets management is critical and easy to get right from the start. Azure Key Vault with the AzureKeyVault@2 task adds only a few lines to the pipeline and completely eliminates the need to store a password in any variable, YAML file, or source code file.
15. Pipeline errors are often environmental, not code errors. The majority of failures in this project were caused by architecture mismatches (amd64 binaries on arm64), missing OS packages (pip3), and permission issues — not bugs in application code.
16. Resource quotas prevent cascading failures. When the guestbook deployment consumed most of the cluster CPU, the monitoring stack could not schedule. ResourceQuota and LimitRange on the microservices namespace ensure one workload cannot starve others.
17. Pipeline loops must be explicitly prevented. Without the [skip ci] tag in the CD pipeline's git commit message, each CD run would trigger a CI run, which would trigger another CD run — an infinite loop that exhausts pipeline minutes.
18. Two Git repositories in the same project requires explicit remote management. Azure DevOps uses its own Git repository for pipeline execution. Both the GitHub remote and the ADO remote must be kept in sync for pipelines to use the latest YAML.